Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 1

Técnicas de Búsqueda Local y Algoritmos Greedy
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como el ratio de mejora relativa del valor:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos empleados cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación de generalización.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$, la calidad de la cartera se mide como:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

donde:

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j$$

siendo $\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^{T} (\mu_{ti} - \bar{\mu}_i)(\mu_{tj} - \bar{\mu}_j)$ la covarianza entre activos i y j, y $\lambda = 500$ el factor de penalización del riesgo.

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i \in 1 \ldots N$$

Los rangos $[lo, hi]$ varían según el conjunto de datos (véase Sección 5).

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución queda representada como un vector $w \in \mathbb{R}^N$ donde cada componente w_i indica la fracción del capital invertida en el activo i. La restricción de cardinalidad implica que w_i puede ser exactamente cero (no se invierte en esa empresa) o estar en el intervalo $[lo, hi]$.

2.2. Generación aleatoria de soluciones

Para construir una solución aleatoria válida se procede como sigue:

1. Se elige aleatoriamente un número de empresas en las que *no* se invierte, $n_0 \in [1, N-1]$.
2. Los índices restantes $n_+ = N - n_0$ se seleccionan barajando un vector de índices.
3. A cada empresa activa se le asigna un peso aleatorio en $[lo, hi]$.
4. Se normaliza: $w_i \leftarrow w_i / \sum_k w_k$.
5. Se llama al operador `fix()` para garantizar factibilidad.

2.3. Operador de reparación (`fix`)

Tras cualquier modificación de la solución puede violarse alguna restricción. El operador de reparación actúa en tres fases:

1. **Normalización:** $w_i \leftarrow w_i / \sum_k w_k$.
2. **Recorte de rango:** si $0 < w_i < lo$ entonces $w_i \leftarrow 0$; si $w_i > hi$ entonces $w_i \leftarrow hi$.
3. **Redistribución iterativa:** mientras $|\sum w_i - 1| > \varepsilon$:
   - Si falta dinero por invertir: se reparte la diferencia entre las empresas activas con $w_i < hi$. Si no hay candidatos, se activa una empresa inactiva asignándole lo.
   - Si sobra dinero: se retira la diferencia de las empresas con $w_i > lo$.

2.4. Precómputo de matrices

Para acelerar la evaluación, se precalculan antes de ejecutar los algoritmos: el vector de factores de beneficio $b_i = \sum_t \log(1 + \mu_{ti})$ y la matriz de covarianza Σ, tanto para los datos de entrenamiento (2015–2024) como para los de test (2025). Así, `fitness` se calcula en $O(N^2)$.

3. Algoritmos

3.1. Algoritmo Aleatorio (*Random Search*)

Genera `MAX_EVALS` soluciones aleatorias independientes y devuelve la mejor. Sirve como referencia inferior de calidad.

Algorithm 1: Búsqueda Aleatoria

Entrada: problema, maxEvals
Salida: mejor solución encontrada

```
mejorFit ← −∞;
para i ← 1 to maxEvals hacer
    s ← crearSolución();
    f ← fitness(s);
    si f > mejorFit entonces
        mejorFit ← f;
        mejorSol ← s;
devolver mejorSol
```

3.2. Algoritmo Greedy

Construye una solución de forma determinista asignando capital en orden decreciente de una función heurística que estima el valor marginal de cada activo.

Función heurística

La heurística para el activo i estima su contribución neta al fitness si fuese el único activo de la cartera, penalizando su covarianza media con el resto:

$$h(i) = b_i - \lambda \cdot \frac{1}{N} \sum_{j=1}^{N} \Sigma_{ij}$$

donde b_i es el factor de beneficio precalculado y Σ_{ij} la covarianza.

Algorithm 2: Greedy

Entrada: problema
Salida: solución construida

```
Calcular h(i) para todo i ∈ {1, ..., N};
Ordenar empresas en orden decreciente por h(i) → lista companies;
w ← 0;   suma ← 0;
mientras suma < 1 − ε y quedan empresas en companies hacer
    c ← primera empresa de companies;
    Eliminar c de companies;
    resto ← 1 − suma;
    si resto < lo entonces
        romper;                    // no se puede añadir más sin violar lo
    w[c] ← mín(hi, resto);
    suma ← suma + w[c];
fix(w);                  // redistribuye si quedan restos por rounding
devolver w
```

El greedy es determinista: produce siempre la misma solución independientemente de la semilla, por lo que su resultado es idéntico en las 50 ejecuciones.

3.3. Búsqueda Local (BL)

Implementa una búsqueda de **primero mejor** (*first improvement*) sobre el vecindario definido por todos los pares de empresas (p_1, p_2), $p_1 \neq p_2$, con $p_1, p_2 \in \{1, \ldots, N\}$.

Operador de generación de vecino

Dado un par $(origen, destino)$, el vecino se obtiene transfiriendo un $ratio = 0{,}4$ del peso del activo origen al activo destino:

$$\text{change}(w, origen, destino): \quad w'_{origen} = w_{origen} \cdot (1 - ratio), \quad w'_{destino} = w_{destino} + ratio \cdot w_{origen}$$

Tras el cambio se aplica `fix()` para restaurar la factibilidad.

Esquema de la BL

Algorithm 3: Búsqueda Local (primero mejor)

Entrada: problema, maxEvals
Salida: mejor solución encontrada

```
P ← todos los pares (i, j) con i ≠ j, i, j ∈ {1, ..., N};
s ← crearSolución(); f ← fitness(s); evals ← 1;
mejora ← verdadero;
mientras mejora y evals < maxEvals hacer
    mejora ← falso;
    barajar(P);
    para cada (origen, destino) ∈ P hacer
        si evals ≥ maxEvals entonces
            romper;
        s' ← s;
        s'[origen] ← s'[origen] · 0,6;
        s'[destino] ← s'[destino] + s[origen] · 0,4;
        fix(s');
        f' ← fitness(s');
        evals += 1;
        si f' > f entonces
            s ← s'; f ← f'; mejora ← verdadero;
            romper;
devolver s
```

El vecindario tiene $N \cdot (N-1)$ vecinos. La exploración se baraja en cada iteración para evitar sesgos de orden. La condición de parada es la convergencia (ninguna mejora en un recorrido completo) o el agotamiento del presupuesto.

4. Estructura del Código y Manual de Usuario

4.1. Estructura de directorios

```
portfolio/
  main.cpp                  # Punto de entrada, experimentos de 50 semillas
  CMakeLists.txt            # Sistema de construcción (CMake)
  boxplot.py                # Script Python para boxplots (pandas + seaborn)
  README.md                 # Manual de usuario y descripción del proyecto
  data/
    ibex_35.csv             # 30 empresas IBEX 35, 2015-2025
    syp_100.csv             # 97 empresas S&P 100, 2015-2025
    syp_500.csv             # 457 empresas S&P 500, 2015-2025
  common/
    mh.h                    # Interfaz genérica MH<tDomain>
    problem.h               # Interfaz genérica Problem<tDomain>
    solution.h              # Typedefs tSolution, tFitness
    random.hpp              # Generador pseudoaleatorio de alta calidad
    util.h                  # Utilidades auxiliares
  inc/
    portfolioProblem.h      # Declaración del problema de portfolio
    randomSearch.h          # Algoritmo aleatorio
    greedy.h                # Algoritmo greedy
    localsearch.h           # Búsqueda local (primero mejor)
  src/
    portfolioProblem.cpp    # Carga de datos, precómputo, fitness, fix
    randomSearch.cpp        # Implementación búsqueda aleatoria
    greedy.cpp              # Implementación greedy
    localsearch.cpp         # Implementación búsqueda local
```

4.2. Compilación

Desde el directorio raíz del proyecto:

```
cmake .
make
```

4.3. Ejecución

El ejecutable `main` acepta opcionalmente dos parámetros:

```
./main [max_evals] [n_seeds]
```

Ejemplos de ejecución:

```
./main                  # valores por defecto: 10000 evaluaciones, 50 semillas
./main 5000             # 5000 evaluaciones, 50 semillas
./main 10000 30         # 10000 evaluaciones, 30 semillas
```

Imprime las tablas de medias por pantalla y genera los ficheros CSV: `results_ibex_35.csv`, `results_syp_100.csv`, `results_syp_500.csv`.

Para generar los boxplots (requiere Python con `pandas` y `seaborn`):

```
python3 boxplot.py
```

Las semillas utilizadas son $1, 2, \ldots,$ `n_seeds`, iguales para todos los algoritmos y conjuntos de datos, garantizando comparaciones justas.

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	**Empresas** (N)	**Días train** (T)	**Días test**	**Rango** $[lo, hi]$
`ibex_35.csv`	30	$\sim$2560	254	[0,005, 0,08]
`syp_100.csv`	97	$\sim$2516	249	[0,005, 0,05]
`syp_500.csv`	457	$\sim$2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, `MAX_EVALS` = 10 000, 50 semillas (1–50).

5.2. Resultados

Las tablas muestran la media de las 50 ejecuciones. El fitness reportado para el Greedy es siempre el mismo al ser determinista.

Cuadro 1: Resultados IBEX 35 — $N = 30$, $T = 2559$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	−0,2128	0,0907	0,1152	1	0,000
Random	−0,5833	0,0910	0,1180	10 000	0,020
BL	−0,2128	0,0907	0,1152	1 852,2	0,001

Cuadro 2: Resultados S&P 100 — $N = 97$, $T = 2516$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	0,8666	−0,0407	0,0099	1	0,000
Random	0,4342	0,0065	0,0518	10 000	0,050
BL	0,8665	−0,0393	0,0111	10 000	0,029

Cuadro 3: Resultados S&P 500 — $N = 457$, $T = 2516$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	1,0430	−0,1157	−0,0628	1	0,000
Random	0,2406	−0,0390	0,0061	10 000	1,193
BL	0,9491	−0,0986	−0,0484	10 000	0,745

5.3. Boxplots

Las Figuras 1–3 muestran la distribución del fitness de entrenamiento en las 50 ejecuciones para cada conjunto de datos.

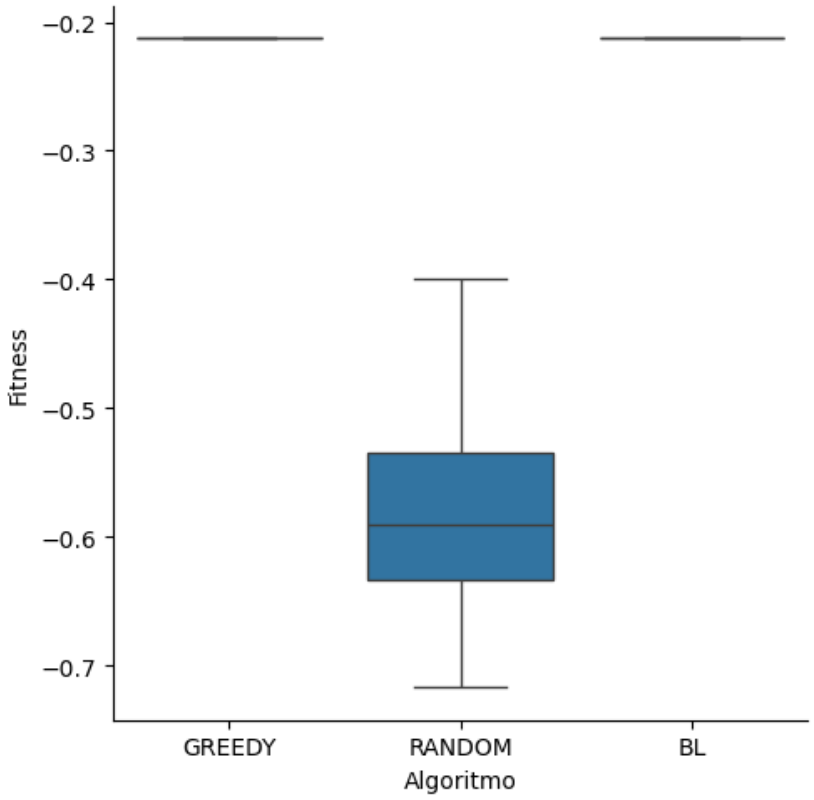


Figura 1: Distribución del fitness (2015–2024) en IBEX 35.

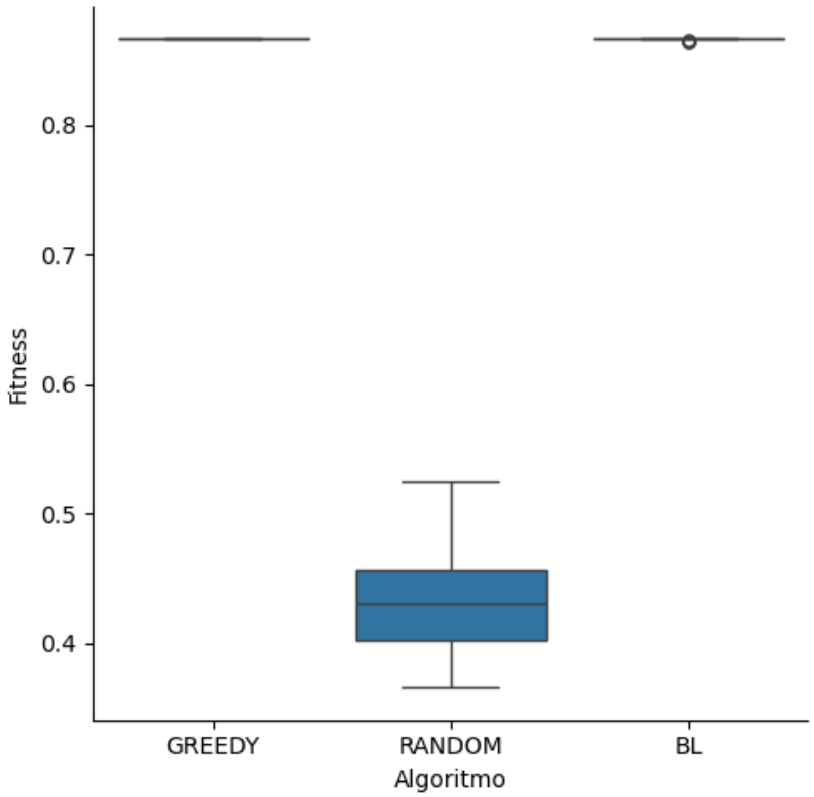


Figura 2: Distribución del fitness (2015–2024) en S&P 100.

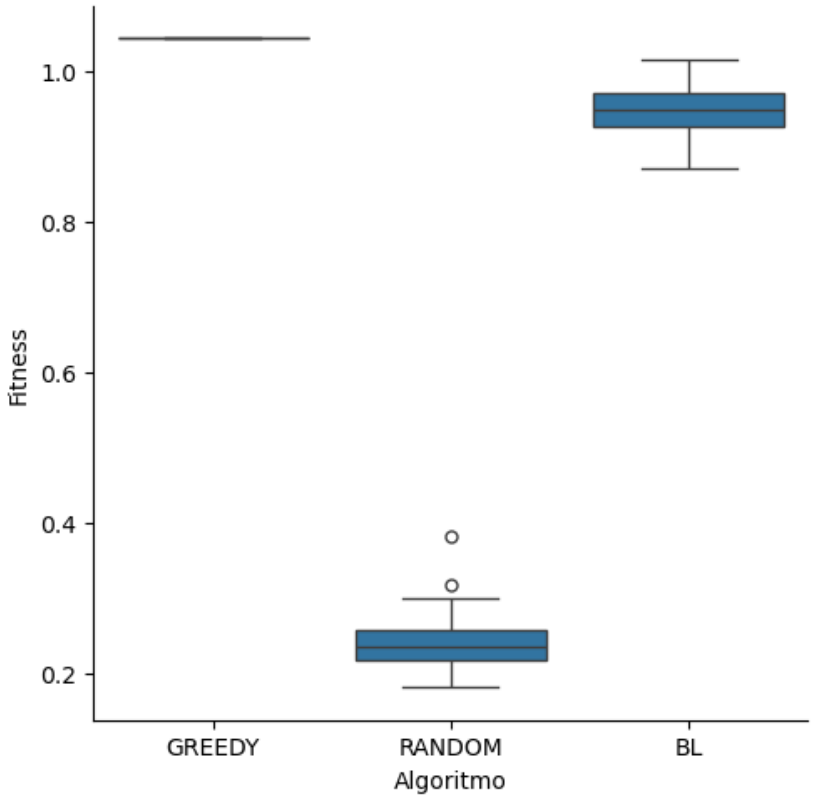


Figura 3: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. Calidad en entrenamiento

Greedy y BL empatan en IBEX 35. Ambos obtienen el mismo fitness de entrenamiento ($-0{,}213$) y el mismo resultado en 2025, lo que indica que el greedy construye directamente el óptimo local al que converge la BL. Esto se confirma en el número de evaluaciones: la BL usa solo 1 852 de las 10 000 disponibles antes de agotar el vecindario sin encontrar mejora, lo que significa que parte de una solución aleatoria y en menos de 2 000 movimientos ya ha llegado a un óptimo local de calidad idéntica a la del greedy.

S&P 100: Greedy y BL idénticos. Con $N = 97$, ambos algoritmos obtienen exactamente el mismo fitness de entrenamiento (0,867). La BL converge al mismo óptimo local que calcula directamente el greedy, usando las 10 000 evaluaciones disponibles.

S&P 500: Greedy supera a BL en entrenamiento (1,043 **vs** 0,949**).** Este resultado, a primera vista sorprendente, se explica por el tamaño del vecindario: con $N = 457$ hay $457 \times 456 = 208\,392$ pares posibles. En un presupuesto de 10 000 evaluaciones, la BL solo puede explorar menos del 5 % del vecindario antes de agotarlos. Partiendo de una solución aleatoria de baja calidad ($\sim 0{,}241$ como muestra Random), la BL asciende cuanto puede, pero no llega al nivel que alcanza el greedy usando información global (beneficio acumulado y covarianza media) para construir su solución en un único paso. Esto ilustra que para instancias grandes, una heurística bien diseñada puede superar a la búsqueda local cuando el presupuesto de evaluaciones es pequeño en relación al tamaño del espacio.

Random es consistentemente el peor en entrenamiento en los tres conjuntos, lo que era esperable: generar 10 000 soluciones aleatorias sin explotar la estructura del problema es la referencia inferior.

6.2. Generalización (datos de 2025)

El comportamiento en test revela un fenómeno notable de **sobreajuste progresivo** con el tamaño del problema:

- **IBEX 35**: todos los algoritmos generalizan bien. Fitness de 2025 positivo ($\approx 0{,}091$) y beneficio positivo ($\approx 0{,}115$–$0{,}118$) para los tres, sin diferencias relevantes.
- **S&P 100**: Greedy y BL presentan fitness negativo en 2025 ($-0{,}041$ y $-0{,}039$), mientras que **Random obtiene el único fitness positivo** ($+0{,}007$) y un beneficio bruto casi cinco veces mayor (0,052 frente a 0,010–0,011).
- **S&P 500**: los tres algoritmos tienen fitness negativo en 2025, y Greedy y BL incluso pierden dinero en bruto (beneficio $-0{,}063$ y $-0{,}048$ respectivamente), mientras que Random logra un beneficio marginalmente positivo ($+0{,}006$).

La razón por la que Random generaliza mejor que los algoritmos más sofisticados es contraintuitiva pero estructural: el greedy y la BL construyen portfolios fuertemente optimizados para la covarianza de 2015–2024. La alta penalización del riesgo ($\lambda = 500$) amplifica este efecto: los algoritmos se dedican a minimizar el riesgo histórico, concentrándose en activos con bajo riesgo relativo en 2015–2024, que en 2025 (un año con condiciones de mercado distintas) resultaron ser peores. Random, al no explotar esa estructura, genera portfolios más diversificados que no dependen de patrones históricos específicos y por eso

transfieren mejor.

6.3. Variabilidad entre semillas

Como es determinista, el **Greedy produce exactamente el mismo resultado** en las 50 ejecuciones (misma solución, mismo fitness). El boxplot del greedy es por tanto un único punto.

La **BL en IBEX 35** presenta también variabilidad muy baja: parte de puntos distintos pero converge al mismo óptimo local (mismo fitness medio que el greedy), con pequeñas variaciones debidas al orden de exploración. En S&P 100 y S&P 500, la BL no converge antes del presupuesto, por lo que su resultado final depende en mayor medida del punto de partida; la varianza es moderada.

Random presenta la mayor varianza en los tres conjuntos, especialmente pronunciada en S&P 500 donde el espacio es enorme y 10 000 muestras representan una fracción ínfima.

6.4. Coste computacional

Dominancia de `fitness`. Cada evaluación de fitness requiere $O(N^2)$ operaciones (producto $w^\top \Sigma w$). Con 10 000 evaluaciones, el coste teórico es $O(10\,000 \cdot N^2)$ operaciones.

Cuadro 4: Tiempo medio por semilla (s) con compilación optimizada (-O3, sin OpenMP)

	IBEX 35 ($N = 30$)	**S&P 100** ($N = 97$)	**S&P 500** ($N = 457$)	**Factor**	**Teórico**
Greedy	0,000	0,000	0,000	—	—
Random	0,020	0,050	1,193	59,7×	$(4{,}7)^2 \approx 22$
BL	0,001	0,029	0,745	744,3×	22

Random y BL escalan de forma casi consistente (60× y 744× entre IBEX 35 y S&P 500). En S&P 500 ambos hacen las 10 000 evaluaciones completas, donde el tiempo está dominado por `fitness`: 1,193 s/semilla para Random y 0,745 s/semilla para BL, una diferencia del 37 %.

Destaca que **en IBEX 35 la BL es 20 veces más rápida que Random** en tiempo promedio (0,001 s frente a 0,020 s): la convergencia temprana (1 852 evaluaciones frente a 10 000) reduce el coste real a solo un 18,5 % del presupuesto.

El **Greedy** es el más eficiente con diferencia: su coste es $O(N^2)$ por el precómputo de heurísticas, pero solo requiere una única evaluación de fitness, resultando en tiempos despreciables incluso para $N = 457$.

Se usó compilación con flags de optimización (`-O3`march=native`ffast-math`) sin paralelización OpenMP.